

Computerarchitektur - Probeklausur

Aufgaben

Aufgabe 1 - Leistungsbewertung

In einer hypothetischen Computer-Architektur wird Multiplikation durch mehrmaliges Addieren durchgeführt, da keine Hardware-Unterstützung für Multiplikation gegeben ist. Für eine derartige Multiplikation werden 180 Taktzyklen benötigt.

a) Wie lautet Amdahls Gesetz? Ein Programm, welches auf einem derartigen Rechnersystem ausgeführt wird, berechnet zu 70 % seiner Ausführungszeit Multiplikationen. Was beträgt die Gesamtbeschleunigung, wenn eine neue hardware-unterstützte Multiplikation verwendet wird, welche in 6 Taktzyklen läuft?

b) Um wie viel muss eine software-basierte Multiplikation den Prozess beschleunigen, um eine Gesamtbeschleunigung von 2,5 zu erreichen? Es werden Programme betrachtet, welche auf dem originalen System 80 % ihrer Laufzeit multiplizieren. Wie viele Taktzyklen läuft die software-basierte Multiplikation?

Aufgabe 2 - Fehlertoleranz

a) Fügen Sie für die nächsten fünf Wörter die erforderlichen Prüf-Bits für den Hamming-Algorithmus ein. Wir arbeiten mit ungerader Parität. Die Lücken sind mit X gekennzeichnet.

Wort-Nr.	1	2	3	4	5	6	7	8	9	10	11	12
1	X	X	1	X	0	1	1	X	0	1	1	0
2	X	X	0	X	1	1	0	X	1	0	0	1
3	X	X	1	X	1	0	0	X	0	1	0	1
4	X	X	0	X	0	1	1	X	1	1	0	0
5	X	X	1	X	0	0	1	X	1	0	1	1

b) Ein nicht fehlertoleranter Code mit Hamming-Abstand 1 besitzt pro Code-Wort 58 Daten-Bits. Wie viele Prüf-Bits sind erforderlich, um ein fehlerhaftes Bit pro Wort zu korrigieren? Begründen Sie.

Aufgabe 3 - Speicherhierarchie

Zwischen CPU und Hauptspeicher eines Rechnersystems liegen die drei Caches L1 bis L3. Auf L1 und L2 wird parallel zugegriffen. Auf L3 und Hauptspeicher wird nach keinem Hit in L1 oder L2 sequentiell zugegriffen.

Zugriffszeiten:

L1: 2 ns

L2: 6 ns

L3: 24 ns

Hauptspeicher: 90 ns

Trefferquoten:

25 % der Speicherzugriffe sind L1-Misses.

80 % der L1-Misses sind L2-Hits.

75 % der L2-Misses sind L3-Hits.

1) Was ist die mittlere Zugriffszeit?

2) Der L3-Cache wird nun entfernt und der Hauptspeicherzugriff beschleunigt. Es soll somit eine schnellere mittlere Zugriffszeit erzeugt werden. Wie schnell muss der neue Hauptspeicher mindestens sein?

Aufgabe 4 - Adressierung

Befüllen Sie folgende Tabellen, welche den Wert eines Akkumulatorregisters, den Inhalt der vier Register R1 bis R4 und die Speicherinhalte nach Lesezugriffen darstellen.

Lesezugriff mit Modus der Adressierung	Wert im Akkumulatorregister
Load Direct 60	140
Load Indirect 220	35
Load Immediate 200	X
Load Indirect-Register-Mode R3	35
Load Register-Mode R2	X
Load Indirect-Register-Mode R4	125
Load Based-Indexed-Mode R1 R2	215
Load Indexed-Mode 80 R1	55
Load Direct 180	X
Load Indirect 240	390

Register

Register	Wert des Registers
R1	X
R2	160
R3	X
R4	X

Speicher

Speicheradresse	Speicherinhalt
60	X
140	390
180	35
220	X
240	X
380	X
420	125
460	215

Aufgabe 5 - Instruktionsabhängigkeit und Pipelining

a) Es gibt 12 Instruktionen (I1 ... I12).

RAW-Abhängigkeiten:

I1 -> I4, I4 -> I5, I5 -> I6, I2 -> I7, I3 -> I8

Die AE arbeiten jede Instruktion in einem Taktzyklus ab.

1) Wie viele Takte braucht man mindestens für alle Instruktionen mit 3 AE, wenn die Reihenfolge fest ist?

2) Wie viele Takte braucht man mit optimaler Ausführung, wenn die Reihenfolge modifiziert werden darf? Instruktionen sollen möglichst frühzeitig ausgeführt werden.

Fügen Sie die Instruktionen in die Tabellen ein. Verwenden Sie jeweils eine Tabelle für 1) und 2).

Tabelle zu 1)

Taktzyklus	AE1	AE2	AE3
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

Aufgabe 5 a) - Fortsetzung

Tabelle zu 2)

Taktzyklus	AE1	AE2	AE3
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

3) Wie viele Takte würden mit 14 AE benötigt? Begründen Sie.

b) Schreiben Sie alle Abhängigkeiten vom Code auf (RAW-, WAW- und WAR-Abhängigkeiten). P sind die Register. Es gilt das Format: INX Pziel, Pquelle1, Pquelle2.

Instruktion	Operanden
I1	INX P11, P21, P31
I2	INX P12, P22, P32
I3	INX P13, P11, P33
I4	INX P14, P12, P13
I5	INX P21, P14, P41
I6	INX P31, P14, P42
I7	INX P11, P14, P43
I8	INX P12, P14, P44

RAW-Abhängigkeiten:

WAR-Abhängigkeiten:

WAW-Abhängigkeiten:

Aufgabe 6 - Mikroarchitektur - Analyse eines ISA-IJVM-Programms

Gegeben sei folgendes ISA-IJVM-Programm als Byte-Code in Hexdarstellung. Die Operanden von ILOAD und ISTORE bezeichnen Wort-Adressen im lokalen Variablenbereich. Die Wort-Adressen in der unteren Tabelle sind im Dezimalsystem angegeben.

Byte-Nr. (dez)	IJVM-Byte-Code	Byte-Nr. (dez)	IJVM-Byte-Code
1	0x15	17	0x08
2	0x0D	18	0x15
3	0x15	19	0x15
4	0x15	20	0x10
5	0x9F	21	0x07
6	0x00	22	0x64
7	0x0D	23	0x36
8	0x15	24	0x34
9	0x0D	25	0x15
10	0x59	26	0x34
11	0x15	27	0x10
12	0x25	28	0x02
13	0x60	29	0x60
14	0x60	30	0x36
15	0xA7	31	0x3B
16	0x00		

Wort-Adresse (dez)	13	21	37	52	59
Variable	A	B	C	X	Y

1) Machen Sie den Code menschenlesbar. Geben Sie die IJVM-Mnemonics mit Variablennamen und die Sprungziele als Byte-Nummern an.

2) Zu Beginn ist der Operandenstapel leer. Bestimmen Sie jeweils, ob der bedingte Sprung ausgeführt wird, und geben Sie die Werte von X und Y nach Programmende an.

a) A = 9, B = 4, C = 6 b) A = 4, B = 4, C = 6

3) Bei welchem Byte wird die maximale Höhe des Operandenstapels erreicht? Wie hoch ist der Stapel maximal? Berücksichtigen Sie beide möglichen Programmpfade.

Aufgabe 7 - Sekundärspeicher - Magnetische Festplatte

Gegeben sei eine Festplatte mit 16 384 Zylindern, 4 Scheiben mit jeweils 2 Oberflächen, einer Rotationsgeschwindigkeit von 6000 UpM, einer mittleren Positionierungszeit von 4 ms und einer Positionierungszeit benachbarter Spuren von 0,5 ms. Die äußeren 8192 Zylinder haben 1024 Sektoren zu je 512 Byte pro Spur. Die inneren 8192 Zylinder haben 512 Sektoren zu je 512 Byte pro Spur.

Mittlere Datenübertragungszeit = Seek Time + Rotational Delay + Transfer Time. Vor dem Lesen der ersten Spur und nach jedem Spurwechsel fällt im Mittel eine halbe Umdrehung als Rotational Delay an.

a) Welche Kapazität hat die Festplatte in GByte?

b) Eine Datei der Größe 2 MiB wird gelesen. Sie liegt in aufeinanderfolgenden Sektoren aufeinanderfolgender Spuren in den äußeren Zylindern. Was ist die mittlere Datenübertragungszeit in Millisekunden?

c) Eine Datei der Größe 2 MiB wird gelesen. Sie liegt aufgeteilt auf je 4 aufeinanderfolgende Sektoren aufeinanderfolgender äußerer Spuren. Was ist die mittlere Datenübertragungszeit in Millisekunden?

Aufgabe 8 - Cache-Kohärenz mit MOESI

Ein paralleles Rechnersystem mit den drei Prozessoren P1, P2 und P3 besitzt je Prozessor einen Cache C1, C2 und C3. Die Prozessoren greifen auf einen gemeinsamen Hauptspeicher CM zu. Es werden das MOESI-Protokoll sowie Write-Back und Write-Allocation verwendet. Für den Hauptspeicher werden die Zustände V (gültig) und I (ungültig) verwendet.

a) Sind die folgenden Zustandskombinationen nach MOESI korrekt? Kreuzen Sie Richtig oder Falsch an.

Nr.	C1	C2	C3	CM	Richtig	Falsch
1	E	I	I	V		
2	S	S	I	V		
3	O	S	I	I		
4	M	I	I	I		
5	M	S	I	I		
6	I	O	S	V		
7	S	I	I	V		

b) Betrachtet werden die Variablen VA und VC in den Cache-Zeilen CLA und CLC. Operationen mit dem Hinweis "gehört zu Takt" finden im selben Takt wie die unmittelbar vorhergehende Operation statt. Vervollständigen Sie die Tabelle mit den Zuständen nach jeder Operation.

Schritt	Operationen	CLA: C1	CLA: C2	CLA: C3	CLA: CM	CLC: C1	CLC: C2	CLC: C3	CLC: CM
vor Beginn		I	I	M	I	E	I	I	V
1	P2: read VA								
gehört zu Takt 1	P3: write VC								
2	P1: write VA								
gehört zu Takt 2	P3: entferne CLC								
3	P2: read VA								
gehört zu Takt 3	P1: read VC								

Hex	Mnemonic	Meaning
0x10	BIPUSH <i>byte</i>	Push byte onto stack
0x59	DUP	Copy top word on stack and push onto stack
0xA7	GOTO <i>offset</i>	Unconditional branch
0x60	IADD	Pop two words from stack; push their sum
0x7E	IAND	Pop two words from stack; push Boolean AND
0x99	IFEQ <i>offset</i>	Pop word from stack and branch if it is zero
0x9B	IFLT <i>offset</i>	Pop word from stack and branch if it is less than zero
0x9F	IF_ICMPEQ <i>offset</i>	Pop two words from stack; branch if equal
0x84	IINC <i>varnum const</i>	Add a constant to a local variable
0x15	ILOAD <i>varnum</i>	Push local variable onto stack
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoke a method
0x80	IOR	Pop two words from stack; push Boolean OR
0xAC	IRETURN	Return from method with integer value
0x36	ISTORE <i>varnum</i>	Pop word from stack and store in local variable
0x64	ISUB	Pop two words from stack; push their difference
0x13	LDC_W <i>index</i>	Push constant from constant pool onto stack
0x00	NOP	Do nothing
0x57	POP	Delete word on top of stack
0x5F	SWAP	Swap the two top words on the stack
0xC4	WIDE	Prefix instruction; next instruction has a 16-bit index

IJVM-Befehlsvorrat

Mit byte, const, varnum als 1-Byte-Operanden
sowie disp, index und offset als 2-Byte-Operanden

Label	Operations	Comments
Main1	PC = PC + 1; fetch; goto (MBR)	MBR holds opcode; get next byte; dispatch
nop1	goto Main1	Do nothing
iadd1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iadd2	H = TOS	H = top of stack
iadd3	MDR = TOS = MDR + H; wr; goto Main1	Add top two words; write to top of stack
isub1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
isub2	H = TOS	H = top of stack
isub3	MDR = TOS = MDR - H; wr; goto Main1	Do subtraction; write to top of stack
iband1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iband2	H = TOS	H = top of stack
iband3	MDR = TOS = MDR AND H; wr; goto Main1	Do AND; write to new top of stack
ior1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
ior2	H = TOS	H = top of stack
ior3	MDR = TOS = MDR OR H; wr; goto Main1	Do OR; write to new top of stack
dup1	MAR = SP = SP + 1	Increment SP and copy to MAR
dup2	MDR = TOS; wr; goto Main1	Write new stack word
pop1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
pop2		Wait for new TOS to be read from memory
pop3	TOS = MDR; goto Main1	Copy new word to TOS
swap1	MAR = SP - 1; rd	Set MAR to SP - 1; read 2nd word from stack
swap2	MAR = SP	Set MAR to top word
swap3	H = MDR; wr	Save TOS in H; write 2nd word to top of stack
swap4	MDR = TOS	Copy old TOS to MDR
swap5	MAR = SP - 1; wr	Set MAR to SP - 1; write as 2nd word on stack
swap6	TOS = H; goto Main1	Update TOS

Mikroprogramm für Mic-1 (Teil 1)

bipush1	SP = MAR = SP + 1	MBR = the byte to push onto stack
bipush2	PC = PC + 1; fetch	Increment PC, fetch next opcode
bipush3	MDR = TOS = MBR; wr; goto Main1	Sign-extend constant and push on stack
iload1	H = LV	MBR contains index; copy LV to H
iload2	MAR = MBRU + H; rd	MAR = address of local variable to push
iload3	MAR = SP = SP + 1	SP points to new top of stack; prepare write
iload4	PC = PC + 1; fetch; wr	Inc PC; get next opcode; write top of stack
iload5	TOS = MDR; goto Main1	Update TOS
istore1	H = LV	MBR contains index; Copy LV to H
istore2	MAR = MBRU + H	MAR = address of local variable to store into
istore3	MDR = TOS; wr	Copy TOS to MDR; write word
istore4	SP = MAR = SP - 1; rd	Read in next-to-top word on stack
istore5	PC = PC + 1; fetch	Increment PC; fetch next opcode
istore6	TOS = MDR; goto Main1	Update TOS
wide1	PC = PC + 1; fetch;	Fetch operand byte or next opcode
wide2	goto (MBR OR 0x100)	Multiway branch with high bit set
wide_ildoad1	PC = PC + 1; fetch	MBR contains 1st index byte; fetch 2nd
wide_ildoad2	H = MBRU << 8	H = 1st index byte shifted left 8 bits
wide_ildoad3	H = MBRU OR H	H = 16-bit index of local variable
wide_ildoad4	MAR = LV + H; rd; goto iload3	MAR = address of local variable to push
wide_istore1	PC = PC + 1; fetch	MBR contains 1st index byte; fetch 2nd
wide_istore2	H = MBRU << 8	H = 1st index byte shifted left 8 bits
wide_istore3	H = MBRU OR H	H = 16-bit index of local variable
wide_istore4	MAR = LV + H; goto istore3	MAR = address of local variable to store into
ldc_w1	PC = PC + 1; fetch	MBR contains 1st index byte; fetch 2nd
ldc_w2	H = MBRU << 8	H = 1st index byte << 8
ldc_w3	H = MBRU OR H	H = 16-bit index into constant pool
ldc_w4	MAR = H + CPP; rd; goto iload3	MAR = address of constant in pool

Mikroprogramm für Mic-1 (Teil 2)

Label	Operations	Comments
iinc1	H = LV	MBR contains index; Copy LV to H
iinc2	MAR = MBRU + H; rd	Copy LV + index to MAR; Read variable
iinc3	PC = PC + 1; fetch	Fetch constant
iinc4	H = MDR	Copy variable to H
iinc5	PC = PC + 1; fetch	Fetch next opcode
iinc6	MDR = MBR + H; wr; goto Main1	Put sum in MDR; update variable
goto1	OPC = PC - 1	Save address of opcode.
goto2	PC = PC + 1; fetch	MBR = 1st byte of offset; fetch 2nd byte
goto3	H = MBR << 8	Shift and save signed first byte in H
goto4	H = MBRU OR H	H = 16-bit branch offset
goto5	PC = OPC + H; fetch	Add offset to OPC
goto6	goto Main1	Wait for fetch of next opcode
iflt1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iflt2	OPC = TOS	Save TOS in OPC temporarily
iflt3	TOS = MDR	Put new top of stack in TOS
iflt4	N = OPC; if (N) goto T; else goto F	Branch on N bit
ifeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
ifeq2	OPC = TOS	Save TOS in OPC temporarily
ifeq3	TOS = MDR	Put new top of stack in TOS
ifeq4	Z = OPC; if (Z) goto T; else goto F	Branch on Z bit
if_jcmpeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
if_jcmpeq2	MAR = SP = SP - 1	Set MAR to read in new top-of-stack
if_jcmpeq3	H = MDR; rd	Copy second stack word to H
if_jcmpeq4	OPC = TOS	Save TOS in OPC temporarily
if_jcmpeq5	TOS = MDR	Put new top of stack in TOS
if_jcmpeq6	Z = OPC - H; if (Z) goto T; else goto F	If top 2 words are equal, goto T, else goto F
T	OPC = PC - 1; goto goto2	Same as goto1; needed for target address
F	PC = PC + 1	Skip first offset byte
F2	PC = PC + 1; fetch	PC now points to next opcode
F3	goto Main1	Wait for fetch of opcode
invokevirtual1	PC = PC + 1; fetch	MBR = index byte 1; inc. PC, get 2nd byte
invokevirtual2	H = MBRU << 8	Shift and save first byte in H
invokevirtual3	H = MBRU OR H	H = offset of method pointer from CPP
invokevirtual4	MAR = CPP + H; rd	Get pointer to method from CPP area
invokevirtual5	OPC = PC + 1	Save Return PC in OPC temporarily
invokevirtual6	PC = MDR; fetch	PC points to new method; get param count
invokevirtual7	PC = PC + 1; fetch	Fetch 2nd byte of parameter count
invokevirtual8	H = MBRU << 8	Shift and save first byte in H
invokevirtual9	H = MBRU OR H	H = number of parameters
invokevirtual10	PC = PC + 1; fetch	Fetch first byte of # locals
invokevirtual11	TOS = SP - H	TOS = address of OBJREF - 1
invokevirtual12	TOS = MAR = TOS + 1	TOS = address of OBJREF (new LV)
invokevirtual13	PC = PC + 1; fetch	Fetch second byte of # locals
invokevirtual14	H = MBRU << 8	Shift and save first byte in H
invokevirtual15	H = MBRU OR H	H = # locals

Mikroprogramm für Mic-1 (Teil 3)

Label	Operations	Comments
invokevirtual16	MDR = SP + H + 1; wr	Overwrite OBJREF with link pointer
invokevirtual17	MAR = SP = MDR;	Set SP, MAR to location to hold old PC
invokevirtual18	MDR = OPC; wr	Save old PC above the local variables
invokevirtual19	MAR = SP = SP + 1	SP points to location to hold old LV
invokevirtual20	MDR = LV; wr	Save old LV above saved PC
invokevirtual21	PC = PC + 1; fetch	Fetch first opcode of new method.
invokevirtual22	LV = TOS; goto Main1	Set LV to point to LV Frame
ireturn1	MAR = SP = LV; rd	Reset SP, MAR to get link pointer
ireturn2		Wait for read
ireturn3	LV = MAR = MDR; rd	Set LV to link ptr; get old PC
ireturn4	MAR = LV + 1	Set MAR to read old LV
ireturn5	PC = MDR; rd; fetch	Restore PC; fetch next opcode
ireturn6	MAR = SP	Set MAR to write TOS
ireturn7	LV = MDR	Restore LV
ireturn8	MDR = TOS; wr; goto Main1	Save return value on original top of stack

Mikroprogramm für Mic-1 (Teil 4)

Label	Operations	Comments
nop1	goto (MBR)	Branch to next instruction
iadd1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iadd2	H = TOS	H = top of stack
iadd3	MDR = TOS = MDR+H; wr; goto (MBR1)	Add top two words; write to new top of stack
isub1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
isub2	H = TOS	H = top of stack
isub3	MDR = TOS = MDR-H; wr; goto (MBR1)	Subtract TOS from Fetched TOS-1
iband1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iband2	H = TOS	H = top of stack
iband3	MDR = TOS = MDR AND H; wr; goto (MBR1)	AND Fetched TOS-1 with TOS
ior1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
ior2	H = TOS	H = top of stack
ior3	MDR = TOS = MDR OR H; wr; goto (MBR1)	OR Fetched TOS-1 with TOS
dup1	MAR = SP = SP + 1	Increment SP; copy to MAR
dup2	MDR = TOS; wr; goto (MBR1)	Write new stack word
pop1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
pop2		Wait for read
pop3	TOS = MDR; goto (MBR1)	Copy new word to TOS
swap1	MAR = SP - 1; rd	Read 2nd word from stack; set MAR to SP
swap2	MAR = SP	Prepare to write new 2nd word
swap3	H = MDR; wr	Save new TOS; write 2nd word to stack
swap4	MDR = TOS	Copy old TOS to MDR
swap5	MAR = SP - 1; wr	Write old TOS to 2nd place on stack
swap6	TOS = H; goto (MBR1)	Update TOS
bipush1	SP = MAR = SP + 1	Set up MAR for writing to new top of stack
bipush2	MDR = TOS = MBR1; wr; goto (MBR1)	Update stack in TOS and memory
iload1	MAR = LV + MBR1U; rd	Move LV + index to MAR; read operand
iload2	MAR = SP = SP + 1	Increment SP; Move new SP to MAR
iload3	TOS = MDR; wr; goto (MBR1)	Update stack in TOS and memory
istore1	MAR = LV + MBR1U	Set MAR to LV + index
istore2	MDR = TOS; wr	Copy TOS for storing
istore3	MAR = SP = SP - 1; rd	Decrement SP; read new TOS
istore4		Wait for read
istore5	TOS = MDR; goto (MBR1)	Update TOS
wide1	goto (MBR1 OR 0x100)	Next address is 0x100 Ored with opcode
wide_ildoad1	MAR = LV + MBR2U; rd; goto iload2	Identical to iload1 but using 2-byte index
wide_istore1	MAR = LV + MBR2U; goto istore2	Identical to istore1 but using 2-byte index
ldc_w1	MAR = CPP + MBR2U; rd; goto iload2	Same as wide_ildoad1 but indexing off CPP

Mikroprogramm für Mic-2 (Teil 1)

iinc1	MAR = LV + MBR1U; rd	Set MAR to LV + index for read
iinc2	H = MBR1	Set H to constant
iinc3	MDR = MDR + H; wr; goto (MBR1)	Increment by constant and update
goto1	H = PC - 1	Copy PC to H
goto2	PC = H + MBR2	Add offset and update PC
goto3		Have to wait for IFU to fetch new opcode
goto4	goto (MBR1)	Dispatch to next instruction
iflt1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iflt2	OPC = TOS	Save TOS in OPC temporarily
iflt3	TOS = MDR	Put new top of stack in TOS
iflt4	N = OPC; if (N) goto T; else goto F	Branch on N bit

Label	Operations	Comments
ifeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
ifeq2	OPC = TOS	Save TOS in OPC temporarily
ifeq3	TOS = MDR	Put new top of stack in TOS
ifeq4	Z = OPC; if (Z) goto T; else goto F	Branch on Z bit
if_icmpeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
if_icmpeq2	MAR = SP = SP - 1	Set MAR to read in new top-of-stack
if_icmpeq3	H = MDR; rd	Copy second stack word to H
if_icmpeq4	OPC = TOS	Save TOS in OPC temporarily
if_icmpeq5	TOS = MDR	Put new top of stack in TOS
if_icmpeq6	Z = H - OPC; if (Z) goto T; else goto F	If top 2 words are equal, goto T, else goto F
T	H = PC - 1; goto goto2	Same as goto1
F	H = MBR2	Touch bytes in MBR2 to discard
F2	goto (MBR1)	
invokevirtual1	MAR = CPP + MBR2U; rd	Put address of method pointer in MAR
invokevirtual2	OPC = PC	Save Return PC in OPC
invokevirtual3	PC = MDR	Set PC to 1st byte of method code.
invokevirtual4	TOS = SP - MBR2U	TOS = address of OBJREF - 1
invokevirtual5	TOS = MAR = H = TOS + 1	TOS = address of OBJREF
invokevirtual6	MDR = SP + MBR2U + 1; wr	Overwrite OBJREF with link pointer
invokevirtual7	MAR = SP = MDR	Set SP, MAR to location to hold old PC
invokevirtual8	MDR = OPC; wr	Prepare to save old PC
invokevirtual9	MAR = SP = SP + 1	Inc. SP to point to location to hold old LV
invokevirtual10	MDR = LV; wr	Save old LV
invokevirtual11	LV = TOS; goto (MBR1)	Set LV to point to zeroth parameter.
ireturn1	MAR = SP = LV; rd	Reset SP, MAR to read Link ptr
ireturn2		Wait for link ptr
ireturn3	LV = MAR = MDR; rd	Set LV, MAR to link ptr; read old PC
ireturn4	MAR = LV + 1	Set MAR to point to old LV; read old LV
ireturn5	PC = MDR; rd	Restore PC
ireturn6	MAR = SP	
ireturn7	LV = MDR	Restore LV
ireturn8	MDR = TOS; wr; goto (MBR1)	Save return value on original top of stack

Mikroprogramm für Mic-2 (Teil 2)